

1 Reinforcement Learning – Imitation Learning – Assignment 1

2 Part A — Training an Expert RL Agent

2.0.1 Library: Stable-Baselines3

[Stable-Baselines3](#) (SB3) is a set of reliable implementations of reinforcement learning algorithms in PyTorch. We use **PPO (Proximal Policy Optimization)**, a popular on-policy algorithm that works well on continuous-control and discrete-action environments alike. PPO strikes a good balance between sample efficiency and stability, making it a natural first choice for LunarLander-v3.

2.0.2 Environment

LunarLander-v3 (discrete actions, 8-dimensional state space, 4 actions):

- State: position (x, y), velocity (vx, vy), angle, angular velocity, two leg contact booleans (r, l).
- Actions: 0 = do nothing, 1 = fire left engine, 2 = fire main engine, 3 = fire right engine.
- Reward: shaped reward for landing between flags, penalises crashing and fuel usage. Solved when the mean reward over 100 episodes exceeds 200.

2.0.3 Performance Criterion

We consider the agent to be a good expert when it achieves a mean episodic reward ≥ 200 over 100 evaluation episodes (the standard Gymnasium benchmark for this environment).

2.1 Training the PPO Agent

We train PPO with:

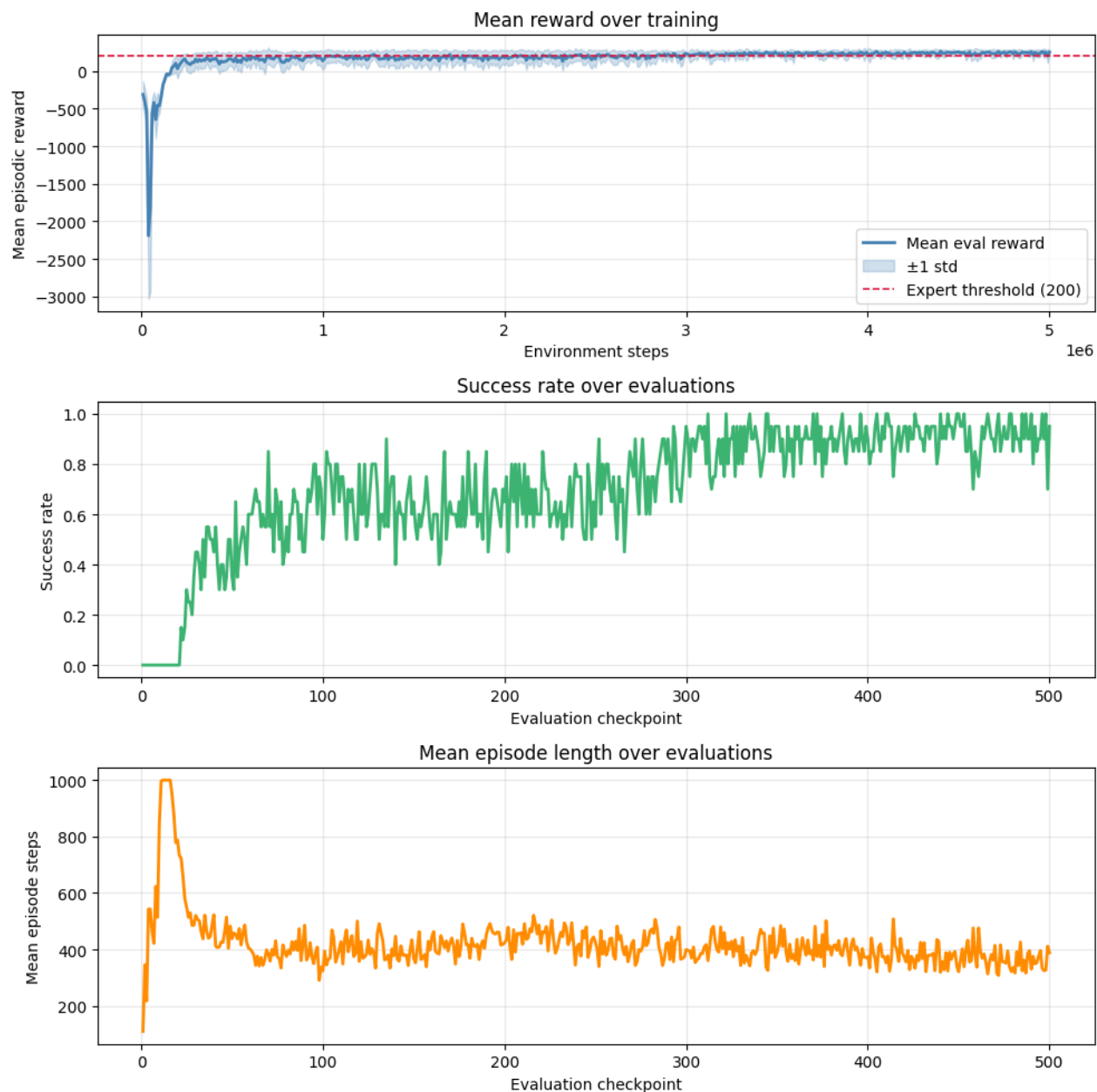
- 4 parallel environments to collect diverse rollouts.
- 5m total timesteps – sufficient for convergence on LunarLander-v3.
- Default SB3 PPO hyperparameters (learning rate $3e-4$, clip range 0.2, GAE $\lambda = 0.95$, $\gamma = 0.99$), which are well-tuned for this task.

An `EvalCallback` runs the current policy in a separate environment every 10k steps and records the mean reward, letting us plot the learning curve.

2.2 Training Progress Verification

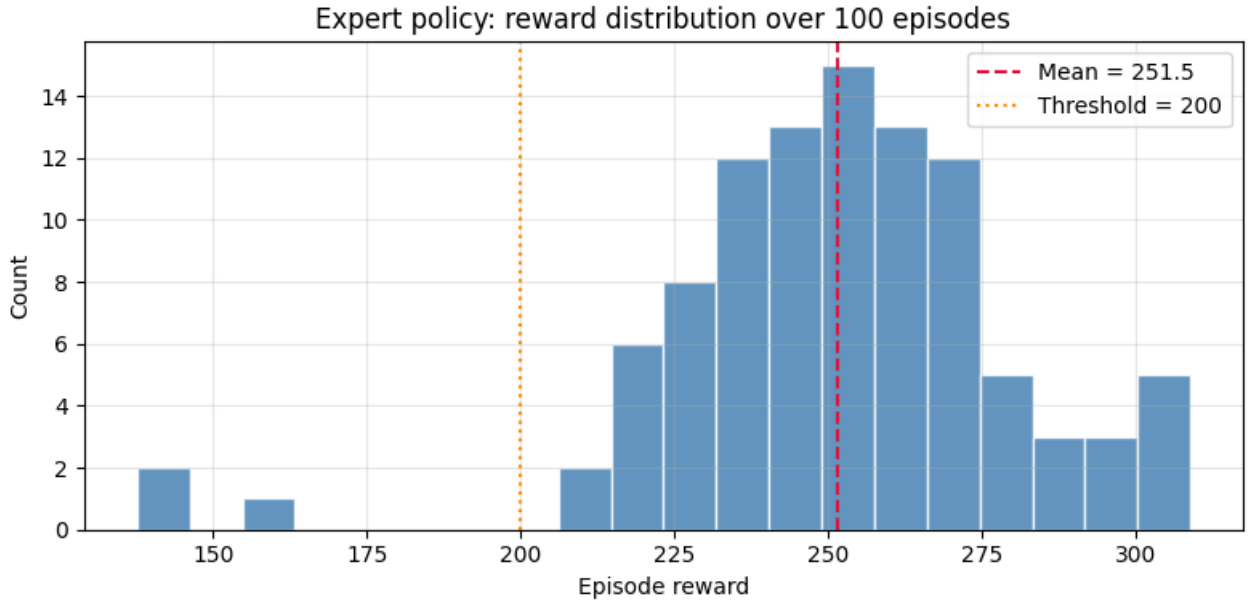
We load the evaluation log saved by `EvalCallback` and plot the following:

- **Mean Reward over time:** to verify that the agent is learning and eventually surpasses the 200 reward threshold.
- **Success Rate over Evaluation Episodes:** to show the percentage of episodes where the agent successfully lands (reward ≥ 200).
- **Mean Episode Length over time:** to observe how the episode length changes as the agent learns to land more efficiently.



2.3 Episode Reward Distribution

A histogram of per-episode rewards gives a fuller picture of the expert's consistency.



Mean: 251.52 | Median: 251.94 | Min: 137.61 | Max: 308.76
 % episodes $\geq$ 200: 97.0%

2.4 Video Demonstration

We record videos of the trained PPO agent in action, showcasing its behaviour across multiple training stages (by number of timesteps). The videos are embedded below for visual inspection.

3 Part B — Imitation Learning

3.1 B1 — Collecting Demonstration Data

We run the trained expert for **200 episodes** and record every (**state**, **action**) pair, yielding a pool of ~70,000 transitions.

From this pool we construct six sub-datasets by random sampling without replacement:

File	Target size
demos_100.npz	100
demos_1000.npz	1,000
demos_5000.npz	5,000
demos_20000.npz	20,000
demos_70000.npz	70,000

Design choice: we sample uniformly at random from the full pool (rather than taking contiguous episode segments) to avoid temporal correlation bias. Episode seeds are fixed for reproducibility. All files are written to data/.

Pool: 70,266 transitions from 200 episodes (mean length: 351 steps)

Action distribution: [10575 13060 31863 14768] (do-nothing / left / main-engine / right)

3.2 B2 — Training an Imitation Model

We treat imitation as 4-class supervised classification task: given a state vector (8 features), predict which of the 4 discrete actions the expert would choose. We train a simple feed forward neural network (MLP) on each dataset and evaluate its performance on a held-out validation set. The reason for this design choice is because it is a straightforward and widely used approach for behaviour cloning, and the simplicity of the model allows us to isolate the effect of dataset size on performance without confounding factors from model complexity.

3.2.1 Architecture — 3-layer MLP

```
ImitationNet(  
  (net): Sequential(  
    (0): Linear(in_features=8, out_features=64, bias=True)  
    (1): GELU()  
    (2): Linear(in_features=64, out_features=64, bias=True)  
    (3): GELU()  
    (4): Linear(in_features=64, out_features=4, bias=True)  
  )  
)
```

3.2.2 Training setup

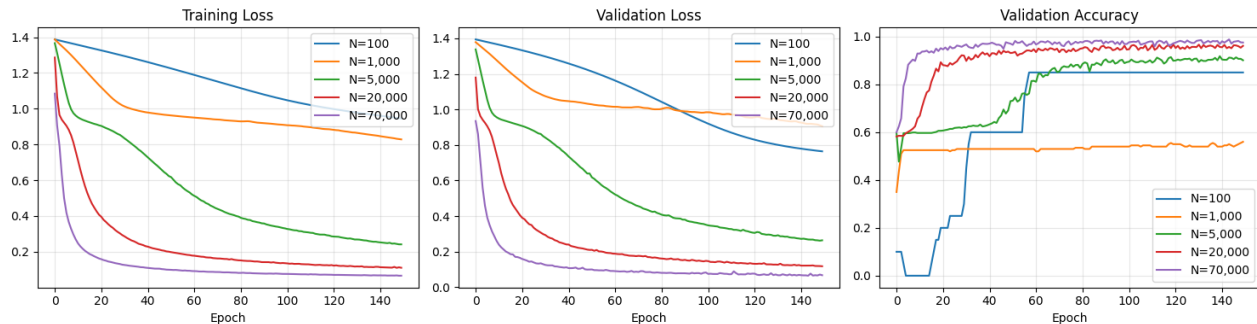
Hyper-parameter	Value
Loss	Cross-entropy
Optimizer	Adam, lr = 1e-3
Batch size	256 (or dataset size if smaller)
Epochs	150
Val split	20 %
Model selection	Best validation loss

The same architecture and hyper-parameters are used for every dataset size so that any performance differences are attributable solely to data quantity.

3.2.3 Training and Validation

We train the model on each dataset and record the training and validation loss and accuracy curves.

N= 100	val_acc=0.850	val_loss=0.7644
N= 1,000	val_acc=0.560	val_loss=0.9058
N= 5,000	val_acc=0.901	val_loss=0.2637
N=20,000	val_acc=0.961	val_loss=0.1177
N=70,000	val_acc=0.975	val_loss=0.0673



3.3 B3 — Evaluating the Imitation Model

We run the best imitator (trained on 70,000 samples) as a policy in LunarLander-v3 and compare it to the expert across 100 episodes.

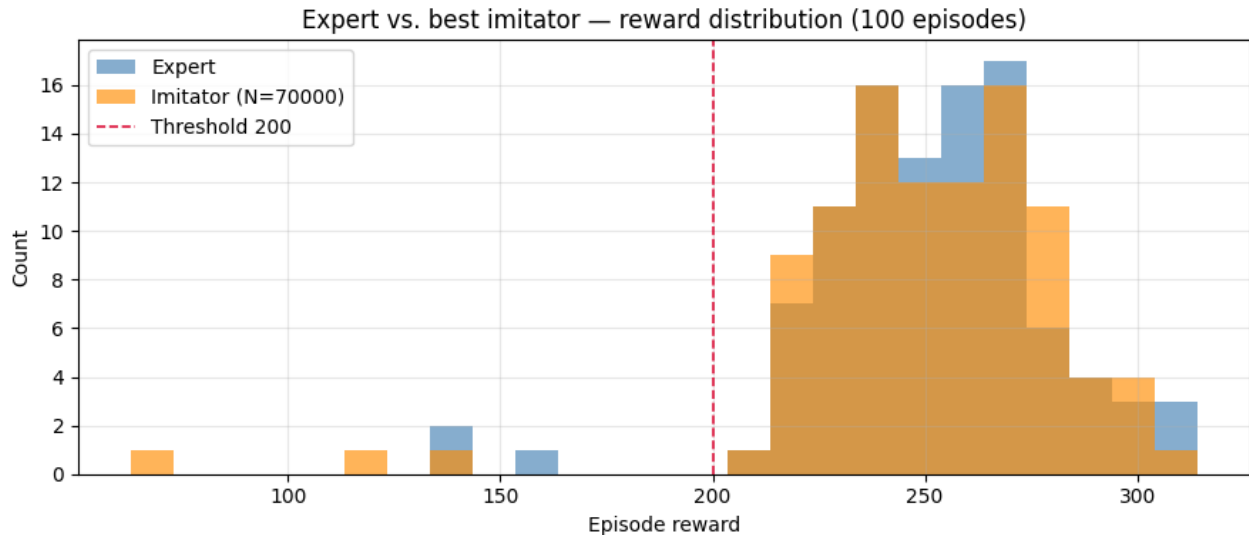
The imitator acts greedily: at each step it feeds the current observation into the network and picks the action with the highest logit (argmax). This is equivalent to a deterministic policy.

Metrics compared:

- Mean Episodic Reward ($\pm$ std)
- Total Success Rate — fraction of episodes with successful landing (reward ≥ 200)
- Landing Pad Success Rate — fraction of episodes where the agent successfully lands on the pad (terminated and $\text{abs}(x) \leq 0.2$)
- Mean Episode Length (proxy for control stability)

Policy	Mean Reward	Std	Total Success %	Landing Pad Success %
Mean Steps				

Expert	251.52	28.66	97.0	96.0
362.9				
Imitator (N=70000)	250.17	33.94	97.0	95.0
344.4				

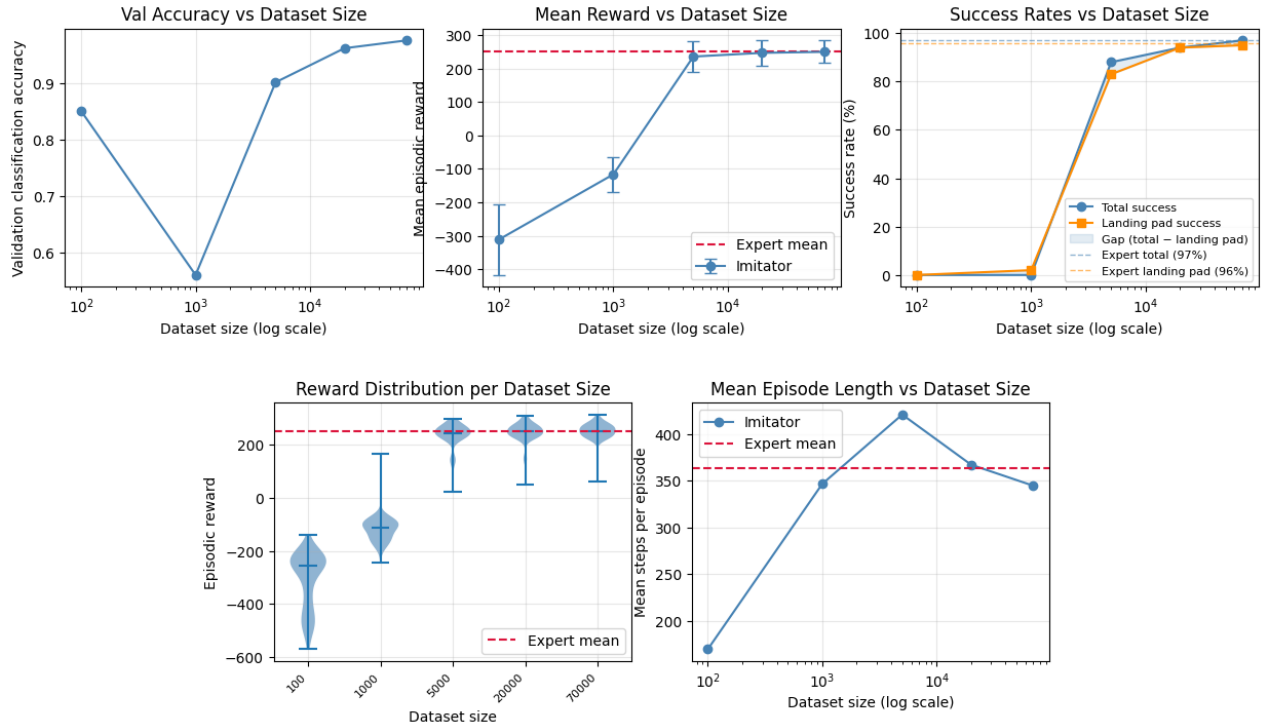


3.3.1 Video Comparison — Expert vs. Best Imitator

We record one episode each (same starting seed) and display them side-by-side so qualitative differences are immediately visible: look for the expert’s smooth, confident descent vs. the imitator’s potential oscillation or drift.

3.4 B4 — Effect of Dataset Size

We train one imitator per dataset size and evaluate each for 100 episodes. This lets us measure how data quantity affects.



- **Val Accuracy vs Dataset Size (top-left):** Classification accuracy on held-out demonstrations rises steeply up to $\sim 5,000$ samples and then plateaus. This reflects the usual supervised learning data efficiency curve: early examples cover new regions of the state space, while later ones are increasingly redundant. Interestingly, the $N=1000$ model underperformed relative to $N=100$, likely due to random sampling variance, although the overall trend still improves with larger datasets.
- **Mean Reward vs Dataset Size (top-center):** Online performance follows a similar trend but is noisier and saturates later. The gap between the imitator and the expert shrinks with more data but never fully closes, a hallmark of behaviour cloning’s distributional-shift problem: small prediction errors compound over a full episode in ways that never appear in offline validation.
- **Success Rates vs Dataset Size (top-right):** At very small N the imitator rarely lands at all. Both success metrics climb together with data, but the gap between *total success* and *landing-pad success* (shaded region) persists across all sizes, indicating that even the best imitator occasionally lands outside the target zone. The expert reference lines show the ceiling we are aiming for.
- **Reward Distribution — Violin (bottom-left):** The violin widths reveal something the mean $\pm$ std cannot: at small N the distribution is bimodal, some episodes happen to go well while others crash early, producing a wide, irregular shape. As N grows the distribution narrows and shifts upward, converging toward a more consistent, expert profile. The median line inside each violin, rising with dataset size confirms the improvement is not driven by a few lucky outliers.
- **Mean Episode Length (bottom-right):** Episode length is a proxy for survival: a crashing agent terminates far earlier than one that attempts a landing. At small N , episodes are short on average — the imitator quickly gets into states it was never trained on and fails. Longer episodes at larger N reflect a policy that at least keeps the lander airborne and manoeuvring, even when it does not score above the success threshold.

3.5 B5 — Challenge: Few-Shot Imitation Learning

Goal: achieve the best imitation performance with the *fewest* training examples.

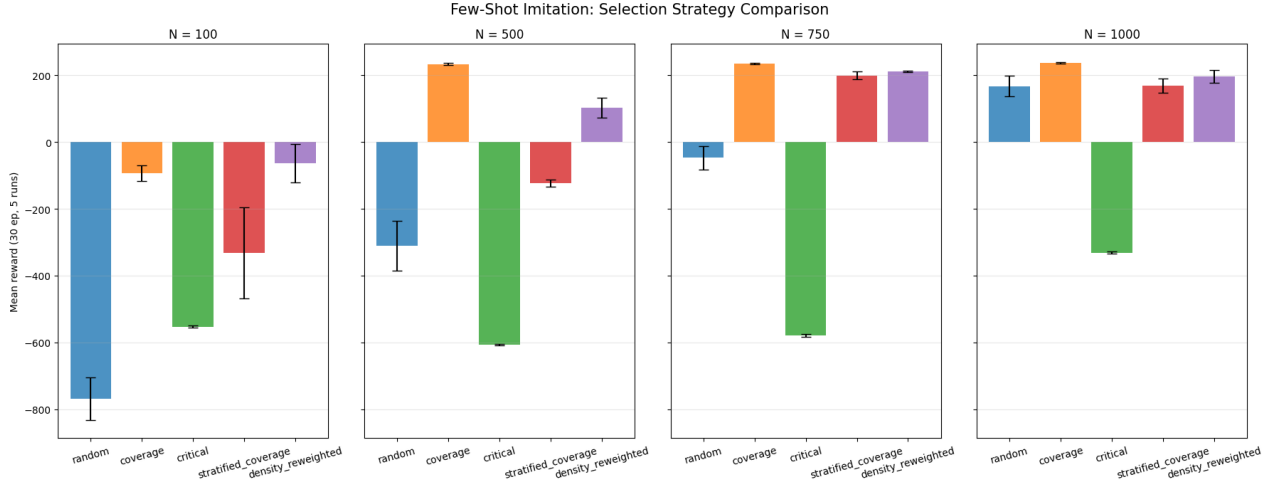
We compare five data-selection strategies applied to the same full demonstration pool:

Strategy	Description
Random	Uniform random sample (baseline)
Coverage	Greedy farthest-point sampling — maximises state-space coverage in normalised feature space
Critical	Select states where the expert is most uncertain (highest action entropy) — the policy’s decision-boundary states
Stratified Coverage	Per-class farthest-point sampling with proportional action-class budgets — coverage with guaranteed class balance
Density-Rewighted Coverage	Farthest-point restricted to the dense core of the pool (top-5% isolated outliers pruned by kNN distance)

Each strategy is evaluated at four dataset sizes (100, 500, 750, 1,000 examples) with 5 independent runs to account for variance.

Hypothesis: Coverage-based and class-balanced strategies should outperform Random at very small N, because they select more *informative and representative* states rather than potentially redundant or imbalanced ones.

random	N= 100	mean= -769.2 +- 64.2
random	N= 500	mean= -310.9 +- 74.5
random	N= 750	mean= -47.0 +- 35.2
random	N=1000	mean= 167.6 +- 30.9
coverage	N= 100	mean= -94.0 +- 23.3
coverage	N= 500	mean= 233.5 +- 3.4
coverage	N= 750	mean= 235.3 +- 2.5
coverage	N=1000	mean= 237.7 +- 2.5
critical	N= 100	mean= -553.2 +- 3.5
critical	N= 500	mean= -606.4 +- 1.7
critical	N= 750	mean= -579.8 +- 4.5
critical	N=1000	mean= -331.0 +- 2.8
stratified_coverage	N= 100	mean= -332.0 +- 137.2
stratified_coverage	N= 500	mean= -124.1 +- 10.7
stratified_coverage	N= 750	mean= 199.6 +- 12.4
stratified_coverage	N=1000	mean= 169.4 +- 21.2
density_reweighted	N= 100	mean= -63.8 +- 57.7
density_reweighted	N= 500	mean= 103.2 +- 29.8
density_reweighted	N= 750	mean= 210.9 +- 2.4
density_reweighted	N=1000	mean= 196.2 +- 19.3



How examples were selected:

Five strategies were compared across dataset sizes of 100, 500, 750, and 1000 samples (5 runs each):

- **Random:** selects a uniform sample from the 200-episode pool with no selection bias, serving as baseline.
- **Coverage:** uses greedy farthest-point sampling in normalised state space, maximally spreading selected states across the 8-dimensional observation space so the imitator sees representative states from all regions.
- **Critical:** selects the top-N states ranked by expert action entropy, targeting the most ambiguous decision-boundary states where the expert is least confident.
- **Stratified Coverage:** runs farthest-point sampling per action class with proportional class budgets (derived from the pool’s action distribution), then merges the results. This ensures every action type is represented while preserving intra-class spatial diversity via PCA-projected state features.
- **Density-Reweighted Coverage:** restricts farthest-point sampling to the dense core of the pool. States in the top 5% of kNN distance (the isolated outlier fringe) are discarded before running coverage, so the budget is not wasted on rare transitions that are unlikely to recur during online rollouts.

What worked well: Plain Coverage was the strongest strategy across all dataset sizes. By greedily maximising spread in normalised state space, it ensures the imitator is exposed to qualitatively different flight conditions even with very few samples, reaching ~ 233 mean reward at $N=500$ while every other strategy was still negative or marginal. Density-Reweighted Coverage was the second-best approach: pruning outlier states before running coverage slightly narrows the gap at larger N ($N=750$, $N=1000$), where both strategies converge to ~ 200 – 210 .

What did not work well: Critical (entropy-based) selection was the worst strategy at every dataset size, consistently achieving strongly negative mean rewards (below -450 even at $N=1000$). Concentrating all training examples near the expert’s decision boundaries starves the imitator of the confident, high-frequency manoeuvres (sustained main-engine thrust and basic orientation control) that constitute the bulk of a successful episode. Stratified Coverage underperformed at small N (100–500): enforcing per-class budgets via a within-class farthest-point search in PCA space restricts each class to a narrow intra-class manifold, losing the global diversity that plain coverage captures. It recovered to competitive performance only at $N \geq 750$, once the per-class budgets were large enough to span the full class geometry. Random sampling improved steadily with N but remained poor at very low N because a small uniform draw has high variance and can miss entire regions of the state space by chance.

Answer: data quantity or quality? At the scale tested, **quality matters more at low N** : with only 100–500 samples, deliberate coverage-based selection dramatically outperforms random sampling. Plain Coverage achieves positive mean reward (~ 233) at $N=500$ while random still averages around -300 . Past $\sim 1,000$ samples the advantage narrows, as a larger random dataset naturally explores sufficient state-space diversity, but even at $N=1000$, Coverage and Density-Reweighted Coverage remain noticeably ahead of random.